

Software Architecture and Design Specification

Project: Traffic Management System (Simulation)

Version: 1.0

Authors: **Team Number:** Team 8

Instructor: Dr. Pradeep Kumar **Team**

Members:

- PES2UG24CS131 — Chetan
- PES2UG24CS159 — Dimpal N
- PES2UG24CS176 — Halahally Shivaprasad Deeksha Prasad
- PES2UG25CS816 — Nikitha V

Date: 15-09-2025

Status: Draft

Revision History

Version	Date	Author	Change Summary
1.0	14-09-2026	Team 8	Initial SAD

Approvals

Role	Name	Signature/Date
Instructor	Dr. Pradeep Kumar	

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Traffic Management System (Simulation), translating the SRS into a modular design for vehicle management, lane queues, traffic signals, emergency priority, monitoring, statistics, validation, reset, and exit.

1.2 Scope

The design covers a standalone console-based simulation of traffic through up to four lanes.

It does not control real-world traffic infrastructure.

1.3 Audience

Developers, testers, instructors, evaluators, and project team members.

1.4 Definitions

TMS—Traffic Management System; SRS—Software Requirements Specification; STP—Software Test Plan; RTM—Requirements Traceability Matrix; Queue—FIFO traffic waiting structure; Lane—independent traffic queue; Emergency Vehicle—priority vehicle; CLI—Command-Line Interface; UML—Unified Modeling Language.

2. Document Overview

2.1 How to use this document

This document presents the architecture, modules, data structures, interfaces, security considerations, sequence flows, error handling, and design rationale. It should be read with the SRS and STP.

2.2 Related Documents

Traffic Management System SRS v1.0; Software Test Plan v1.0; Requirements Traceability Matrix.

3. Architecture

3.1 Goals & Constraints

Goals: modularity, correct queue behavior, safe input handling, predictable response, maintainability, and clear CLI interaction.

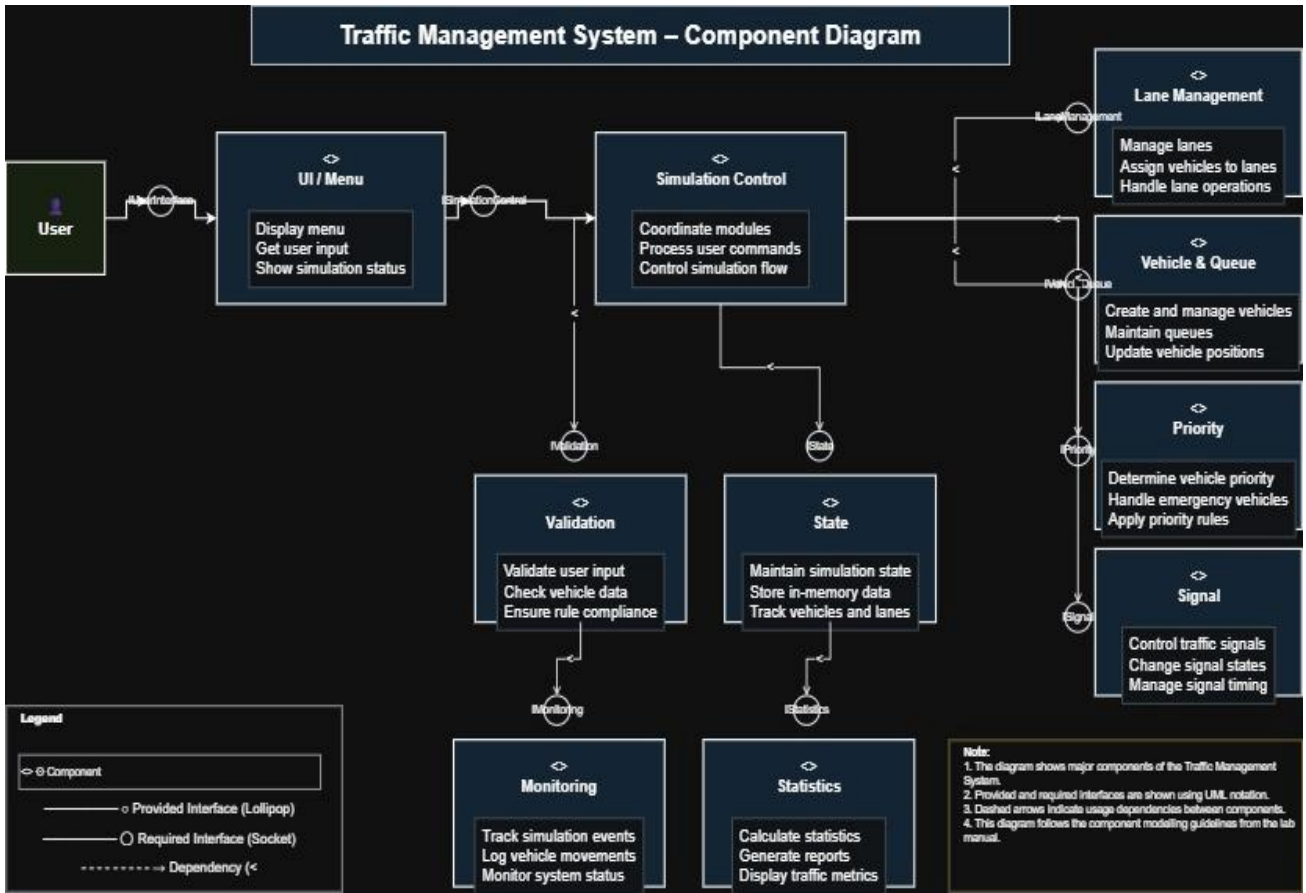
Constraints: C language, console execution, maximum four lanes, finite queue capacity, no database/network service, simulation only.

3.2 Stakeholders & Concerns

Stakeholder	Concerns
System User	Simple interaction and correct processing
Instructor/Evaluator	Requirement coverage and design clarity
Developer	Modularity and maintainability

3.3 Component (UML) Diagram

The following UML component diagram shows the main logical components and their relationships.



3.4 Component Descriptions

Component	Responsibilities
UI/Menu	Display menu, collect commands, show results
Vehicle/Queue	Add, process, and display vehicles
Lane Management	Validate and isolate lanes
Priority	Handle emergency vehicle priority

Signal	Set/display Red or Green
Monitoring	Calculate density and queue size
Statistics	Track added/processed totals
Validation	Validate input and bounds
Simulation Control	Reset state and exit

3.5 Chosen Architecture Pattern and Rationale

A modular layered architecture is chosen. The presentation layer handles CLI interaction; the control layer coordinates operations; domain/data modules implement vehicles, queues, priority, signals, monitoring, and statistics; validation provides a safety boundary. This is simpler and more maintainable than a distributed or microservices architecture for a standalone simulation.

3.6 Technology Stack & Data Stores

Area	Choice
Language	C
Execution	Console application
Data structures	Structures and queue-based storage
Build	Standard C compiler such as GCC
Persistence	None; in-memory simulation state
External services	None

3.7 Risks & Mitigations

Risk	Mitigation
Invalid input	Validate before access
Queue overflow	Check capacity
Queue underflow	Check empty state
Wrong priority	Dedicated priority logic and tests
State inconsistency	Centralize updates and reset

3.8 Traceability to Requirements

Vehicle/Queue → TMS-F-001-003; Lane Management → TMS-F-004-006; Signal/Priority → TMS-F-007-009; Monitoring/Statistics → TMS-F-010-012; Control/Validation → TMS-F-013-015; Quality/security → TMS-NF-001-005 and TMS-SR-001-005.

3.9 Security Architecture

- Validate lane indices before access.
- Enforce queue capacity before insertion.

- Validate vehicle identifiers.
- Treat input only as simulation data; do not execute OS commands.
- Do not collect or display confidential personal information.

4. Design

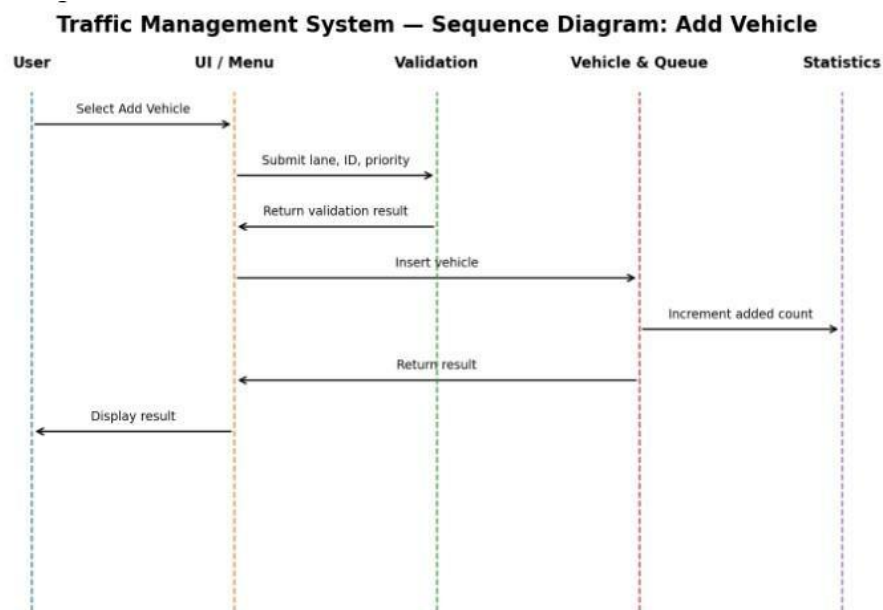
4.1 Design Overview

The system initializes simulation state, presents a CLI menu, validates the selected operation, updates in-memory state through the relevant module, and displays the result.

Each lane has an independent queue and emergency vehicles receive priority.

4.2 UML Sequence Diagrams

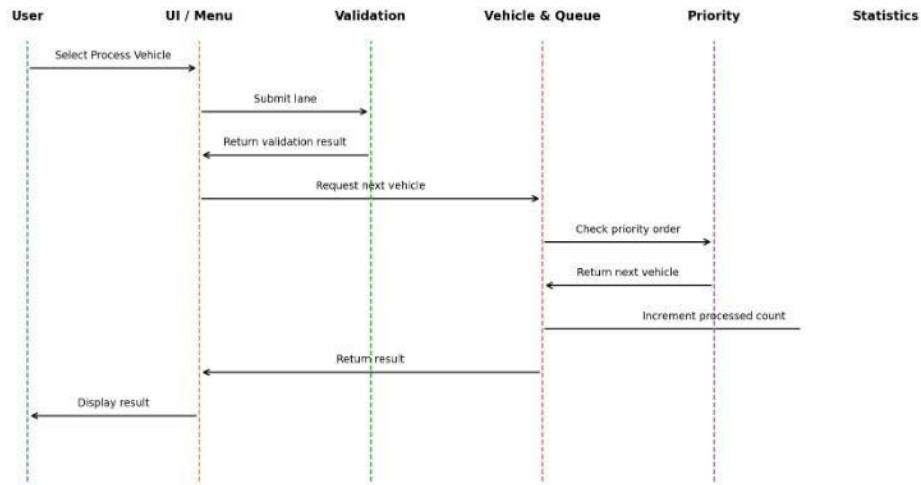
Sequence Diagram 1 — Add Vehicle



Step	Component	Interaction
1	User	Select Add Vehicle
2	UI	Request lane, ID, priority
3	Validation	Check input/capacity
4	Queue/Priority	Insert vehicle
5	Statistics	Increment added count
6	UI	Display result

Sequence Diagram 2 — Process Vehicle

Traffic Management System — Sequence Diagram: Process Vehicle



Step	Component	Interaction
1	User	Select Process Vehicle
2	UI	Request lane
3	Validation	Check lane
4	Queue/Priority	Select and remove next vehicle
5	Statistics	Increment processed count
6	UI	Display result

4.3 API Design

This is a standalone C application and exposes no network API. Internal interfaces are specified for design:

Interface	Operation	Inputs	Effect
Queue	addVehicle()	lane, vehicle	Insert if valid/capacity available
Queue	processVehicle()	lane	Remove/process next vehicle
Monitoring	getDensity()	lane	Return density class
Signal	setSignal()	state	Update signal state

4.4 Error Handling, Logging & Monitoring

- Reject invalid menu/lane values without crashing.
- Reject insertion into full queues.
- Handle processing of empty queues safely.
- Reject invalid vehicle IDs.
- Do not display confidential information.
- Display queue size, density, signal status, and statistics.

4.5 UX Design

The CLI uses a numbered menu, clear prompts, concise success/error messages, and explicit status output.

4.6 Open Issues & Next Steps

- Insert finalized UML component and sequence diagrams as graphics.
- Verify exact queue capacity/density thresholds against the implemented code if specified separately.
- Complete STP and execute mapped tests.

5. Appendices

5.1 Glossary

TMS, SRS, STP, RTM, CLI, UML.

5.2 References

Traffic Management System SRS v1.0; Software Test Plan template; SAD template.

5.3 Tools

C compiler (e.g., GCC), Git/GitHub, PlantUML or draw.io, command-line environment.